

```
medikondarakesh@Medikondas-MacBook-Air ~ % nano shell_parser.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc shell_parser.c -o shell_parser
medikondarakesh@Medikondas-MacBook-Air ~ % ./shell_parser
```

```
Parser Shell > ls -l > output.txt
--- Debug: Tokenizer Output ---
Token [0]: Value="ls", Type=0
Token [1]: Value="-l", Type=0
Token [2]: Value=">", Type=3
Token [3]: Value="output.txt", Type=0
--- Debug: Parser & Execution Structure ---
Command Executable: "ls"
Arguments Count: 2
Redirect Output To: output.txt
```

```
Parser Shell > ls -l >
--- Debug: Tokenizer Output ---
Token [0]: Value="ls", Type=0
Token [1]: Value="-l", Type=0
Token [2]: Value=">", Type=3
--- Debug: Parser & Execution Structure ---
[Syntax Error] Missing output redirection file after '>'
```

```
Parser Shell > | grep test
--- Debug: Tokenizer Output ---
Token [0]: Value="|", Type=1
Token [1]: Value="grep", Type=0
Token [2]: Value="test", Type=0
--- Debug: Parser & Execution Structure ---
[Syntax Error] Unexpected token '|' at start of command.
```

```
Parser Shell > cat input.txt | grep error
--- Debug: Tokenizer Output ---
Token [0]: Value="cat", Type=0
Token [1]: Value="input.txt", Type=0
Token [2]: Value="|", Type=1
Token [3]: Value="grep", Type=0
Token [4]: Value="error", Type=0
--- Debug: Parser & Execution Structure ---
Command Executable: "cat"
Arguments Count: 4
Pipeline Flag: Active
```

```
Parser Shell > exit
medikondarakesh@Medikondas-MacBook-Air ~ %
```

```
    }
    1++;
}

Get Help    WriteOut    Read File    Prev Pg    Cut Text    Cur Pos
Exit        Justify      Where is     Next Pg     UnCut Text  To Spell

UW PICO 5.09                                     File: ar.c                                     Modified

char *input_file;      // Name of input file for redirection '<'[cite: 1]
char *output_file;     // Name of output file for redirection '>'[cite: 1]
int has_pipe;          // Flag indicating if command includes a pipe '|'[cite: 1]
} Command;

// Tokenizer: Splits user input into tokens based on whitespace and special delimiters[cite: 2]
int tokenize(const char *input, Token tokens[]) {
    int token_count = 0;
    int len = strlen(input);
    int i = 0;

    while (i < len) {
        // Handle whitespace: skip spaces, tabs, and newline characters[cite: 2]
        if (isspace((unsigned char)input[i])) {
            i++;
            continue;
        }

        // Identify Delimiters: '|', '<', '>'[cite: 2]
        if (input[i] == '|') {
            tokens[token_count].value = strdup("|");
            tokens[token_count].type = TOKEN_PIPE;
            token_count++;
            i++;
        }
        else if (input[i] == '<') {
            tokens[token_count].value = strdup("<");
            tokens[token_count].type = TOKEN_REDIRECT_IN;
            token_count++;
            i++;
        }
        else if (input[i] == '>') {
            tokens[token_count].value = strdup(">");
            tokens[token_count].type = TOKEN_REDIRECT_OUT;
            token_count++;
            i++;
        }
        else {
            // Read Word / Argument: scan until whitespace or a delimiter[cite: 2]
            int start = i;
            while (i < len && !isspace((unsigned char)input[i]) &&
                input[i] != '|' && input[i] != '<' && input[i] != '>') {
                i++;
            }

            int word_len = i - start;
            char *word = malloc(word_len + 1);
            strncpy(word, input + start, word_len);
            word[word_len] = '\0';

            tokens[token_count].value = word;
        }
    }
}
```

UW PICO 5.09

File: ar.c

Modified

```
tokens[token_count].value = strdup("<");
tokens[token_count].type = TOKEN_REDIRECT_IN;
token_count++;
i++;
}
else if (input[i] == '>') {
tokens[token_count].value = strdup(">");
tokens[token_count].type = TOKEN_REDIRECT_OUT;
token_count++;
i++;
}
else {
// Read Word / Argument: scan until whitespace or a delimiter[cite: 2]
int start = i;
while (i < len && !isspace((unsigned char)input[i]) &&
input[i] != '|' && input[i] != '<' && input[i] != '>') {
i++;
}

int word_len = i - start;
char *word = malloc(word_len + 1);
strncpy(word, input + start, word_len);
word[word_len] = '\0';

tokens[token_count].value = word;
tokens[token_count].type = TOKEN_WORD;
token_count++;
}
}
return token_count; // Return total token count[cite: 2]
}

// Parser: Validates syntax and populates execution structures[cite: 2]
int parse(Token tokens[], int token_count, Command *cmd) {
// Handle empty commands[cite: 2]
if (token_count == 0) {
printf("[Parser] Empty command detected.\n");
return 0; // Valid, but nothing to execute[cite: 2]
}

// Syntax Error check: Command starts with a pipe[cite: 2]
if (tokens[0].type == TOKEN_PIPE) {
printf("[Syntax Error] Unexpected token '|' at start of command.\n");
return -1;
}

for (int i = 0; i < token_count; i++) {
if (tokens[i].type == TOKEN_WORD) {
cmd->args[cmd->arg_count++] = tokens[i].value;
}
else if (tokens[i].type == TOKEN_REDIRECT_OUT) {
```

⌘ Get Help

⌘ Exit

⌘ WriteOut

⌘ Justify

⌘ Read File

⌘ Where is

⌘ Prev Pg

⌘ Next Pg

⌘ Cut Text

⌘ UnCut Text

⌘ Cur Pos

⌘ To Spell

UW PICO 5.09

File: ar.c

Modified

```
int token_count = tokenize(input_buf, tokens);

// Debug Output: Tokens[cite: 4]
printf("---- Debug: Tokenizer Output ----\n");
for (int i = 0; i < token_count; i++) {
printf("Token [%d]: Value=\"%s\", Type=%d\n", i, tokens[i].value, tokens[i].type);
}

// Step B: Parse & Validate
printf("---- Debug: Parser & Execution Structure ----\n");
int status = parse(tokens, token_count, &cmd);

if (status == 1) {
printf("Command Executable: \"%s\"\n", cmd.args[0]);
printf("Arguments Count: %d\n", cmd.arg_count);
if (cmd.output_file) printf("Redirect Output To: %s\n", cmd.output_file);
if (cmd.input_file) printf("Redirect Input From: %s\n", cmd.input_file);
if (cmd.has_pipe) printf("Pipeline Flag: Active\n");
}

// Step C: Free token memory
free_tokens(tokens, token_count);
}

return 0;
}


```

⌘ Get Help

⌘ Exit

⌘ WriteOut

⌘ Justify

⌘ Read File

⌘ Where is

⌘ Prev Pg

⌘ Next Pg

⌘ Cut Text

⌘ UnCut Text

⌘ Cur Pos

⌘ To Spell